HANGFUL

Software Design Description (SDD)

&

Database Design Description (DBDD)

CMSI 4072: Senior Project II
Loyola Marymount University
Spring 2026

Version 2.0 | February 2026 | Scoped MVP

Tailoring Note

This document combines the Software Design Description (SDD) and Database Design Description (DBDD) into a single unified manuscript. The SDD sections (6.1 through 6.3) present the software architecture and detailed design, while section 6.4 contains the database design and description that would typically appear in a separate DBDD document.

Scope Note: This SDD describes the semester-deliverable MVP scope of Hangful, which consists of two components: an iOS User App (Swift/SwiftUI) and a Backend API (Node.js/Express + PostgreSQL).

6.1 Introduction

This document presents the architecture and detailed design for the software and database systems of the Hangful project. Hangful is a social utility iOS application designed to reduce friction in planning casual hangouts among Gen Z users (ages 18–28) by combining deal discovery with social coordination. Users browse promotional deals from local brands, claim deals, attend hangouts with friends, upload photographic proof, and receive unique redemption codes as rewards. A referral system incentivizes organic growth by unlocking shareable magic links after a user's first completed hangout.

The semester-deliverable MVP consists of two components: a native iOS User App built with Swift 5.9 and SwiftUI following the MVVM architectural pattern, and a Backend API server built with Node.js and Express backed by a PostgreSQL database on Supabase. Proof images are stored in AWS S3, and user authentication uses Twilio's Verify API for phone-based OTP. Brand and deal management is handled through database seeding and lightweight admin endpoints rather than a full web portal, keeping the scope achievable as a solo project.

6.1.1 System Objectives

The objective of the Hangful application is to provide a frictionless, mobile-first platform that incentivizes real-world social interaction among close friends through brand-sponsored deals. The MVP validates the core product loop: User signs up → User browses deals → User claims a deal → User hangs out with friends → User uploads proof → Admin approves proof → User receives redemption code → User shares referral link → New users join.

Specifically, the MVP aims to achieve the following objectives. First, allow users to authenticate securely via phone number and one-time password, browse a feed of available deals, and claim deals with a single tap while enforcing quantity limits through atomic database transactions. Second, enable users to upload photographic proof of their hangout and receive a unique, cryptographically random redemption code upon admin approval. Third, implement a referral system gated behind the first completed hangout that generates shareable magic links for viral growth. Fourth, deliver SMS reminders for claimed deals approaching expiration to drive claim-to-redemption conversion. Fifth, provide a brand request submission feature that captures user demand signals for future brand partnerships.

6.1.2 Hardware, Software, and Human Interfaces

6.1.2.1 iOS Mobile Device Interface

The primary hardware interface is the Apple iPhone running iOS 17.0 or later. Required hardware capabilities include: a touchscreen display for all user interactions including deal browsing, claiming, and navigation; a rear-facing camera (minimum 8MP) for capturing proof-of-hangout photographs; and cellular or Wi-Fi network connectivity for API communication and image uploads. The application is developed natively using Swift 5.9 and SwiftUI with the Combine framework for reactive data binding, targeting a minimum device of iPhone SE (2nd generation). The app uses the iOS Keychain for secure JWT token storage and UIImagePickerController for camera and photo library access.

6.1.2.2 Backend API Server

The backend API server runs on Node.js 2.0 LTS with the Express 4 framework, hosted on Railway with auto-scaling capabilities. The server communicates with the iOS client exclusively over HTTPS using TLS 1.3 certificates managed by Cloudflare. The RESTful API follows JSON request/response conventions and uses JWT (JSON Web Tokens) for stateless authentication via the jsonwebtoken library (version 9.x). All API endpoints except authentication routes require a valid Bearer token in the Authorization header. Rate limiting is enforced on OTP requests (maximum 3 per phone number per 10 minutes) via the express-rate-limit middleware.

6.1.2.3 PostgreSQL Database Server

The application uses PostgreSQL 16 hosted on Supabase as its primary relational data store. Database access is managed through the Prisma ORM (version 5.x), which provides type-safe query generation, schema migration management, and connection pooling via PgBouncer. The database server accepts connections only from the backend API server via authenticated SSL connection strings; no direct client-to-database communication is permitted. The schema consists of four tables (users, deals, claims, brand_requests) with UUID primary keys, ENUM constraints, and foreign key relationships enforcing referential integrity.

6.1.2.4 Object Storage Service (AWS S3)

Proof-of-hangout images are stored in an AWS S3 bucket. The iOS app uploads images directly to S3 using time-limited pre-signed URLs generated by the backend API, bypassing the API server for large file transfers. Images are served through Amazon CloudFront CDN with Cache-Control headers for optimized delivery. Maximum upload size is 10MB per image; the iOS app compresses images to JPEG at 0.7 quality (targeting under 1MB) before upload. The S3 bucket is configured with CORS headers permitting PUT requests from the app and IAM policies restricting access to the API server's credentials.

6.1.2.5 Twilio SMS Services

The Twilio Verify API (version 2) handles one-time password delivery for user authentication. When a user requests an OTP, the backend calls Twilio's /Verifications endpoint to send a 6-digit code via SMS; upon user entry, the backend calls /VerificationCheck to validate. The Twilio Messaging API handles transactional SMS notifications including deal expiration reminders (24 hours and 2 hours before expiry) and proof approval/rejection notifications. Twilio credentials (ACCOUNT_SID and AUTH_TOKEN) are stored as environment variables on Railway and are never committed to source control.

6.1.2.6 Third-Party Libraries

Library	Platform	Purpose and Usage in Hangful
Alamofire 5.x	iOS	HTTP networking library used by the APIClient service for all REST API communication with the backend; provides request/response serialization, automatic retry, and request interceptors for JWT token injection
KeychainSwift 22.x	iOS	Wrapper around the iOS Keychain used by KeychainManager for secure storage of JWT access and refresh tokens with kSecAttrAccessibleWhenUnlockedThisDeviceOnly protection
SDWebImageSwiftUI 3.x	iOS	Asynchronous image loading library used by DealsView and DealDetailView for loading brand logos and deal images from the CloudFront CDN with memory and disk caching
Prisma 5.x	Backend	Type-safe ORM used for all database operations including schema migrations, query building with parameterized inputs (preventing SQL injection), interactive transactions for atomic deal claiming, and connection pooling
jsonwebtoken 9.x	Backend	JWT library used by auth middleware for signing tokens on login (HS256 algorithm, 15-min access / 7-day refresh expiry) and verifying tokens on every protected API request
Multer 1.x	Backend	Multipart form data parsing middleware used by the proof upload endpoint for handling image file uploads before forwarding to S3 via pre-signed URLs
bcrypt 5.x	Backend	Password hashing library reserved for future brand portal authentication; included in the dependency tree for admin endpoint password protection with cost factor 12
express-rate-limit 7.x	Backend	Rate limiting middleware applied to OTP request endpoint (3 requests per phone per 10 minutes) and claim endpoint (10 claims per user per minute) to prevent abuse

6.1.2.7 Human Interface

The iOS User App presents a tab-based navigation interface with three primary sections: a Deal Feed showing all available deals in a vertically scrolling card layout with brand logo, title, reward type, and remaining quantity; a My Claims section listing the user's claimed deals with color-coded status chips (claimed, proof pending, approved, rejected, redeemed); and a Profile section containing the referral link sharing feature (unlocked after first completed hangout), brand request submission form, and account information. Each deal card taps through to a DealDetailView with full description, terms, and a prominent claim button. The proof upload flow presents the device camera or photo library via UIImagePickerController with a preview and upload progress indicator.

6.2 Architectural Design

The Hangful MVP follows a two-tier client-server architecture consisting of a presentation layer (iOS App) and a combined business logic and data layer (Backend API + PostgreSQL + S3). The iOS app handles all user interaction, local state management, and image capture, while the backend API serves as the single source of truth for all application state, enforces business rules (quantity limits, claim uniqueness, referral eligibility), and coordinates with external services (Twilio for SMS, S3 for storage). This separation ensures the iOS app contains no business logic beyond input validation and presentation, making the backend independently testable and the iOS app a pure consumer of the API contract.

6.2.1 Major Software Components

iOS User App (Swift/SwiftUI)

The iOS User App is the sole consumer-facing interface for the MVP. It implements the MVVM (Model-View-ViewModel) architectural pattern where SwiftUI Views declaratively render UI based on @Published state from ObservableObject ViewModels, which delegate data operations to Service classes. The app contains four Model structs (User, Deal, Claim, BrandRequest), eight View files (AuthView, DealsView, DealDetailView, MyClaimsView, ProofUploadView, RedemptionView, ReferralView, RequestBrandView), four ViewModel classes (AuthViewModel, DealsViewModel, ClaimsViewModel, ReferralViewModel), and four Service classes (APIClient, AuthService, ImageUploader, KeychainManager). This component satisfies functional requirements UR-001 (phone OTP auth), UR-002 (deal feed), UR-003 (single-tap claiming), UR-004 (proof photo upload), UR-005 (redemption code display), UR-006 (referral magic link), UR-007 (SMS reminder reception), and UR-009 (brand request submission).

Backend API Server (Node.js/Express)

The Backend API is the central business logic layer that processes all client requests, enforces business rules, and coordinates between the database, object storage, and Twilio. It is organized into five directories: routes/ (endpoint definitions), controllers/ (request handling and response formatting), services/ (business logic including claim validation, code generation, and SMS scheduling), middleware/ (JWT auth verification, rate limiting, input validation), and models/ (Prisma schema and generated client). The API exposes 10 endpoints across four route groups: Authentication (request-otp, verify-otp), Deals (list all, claim), Proofs (upload, check status), Referrals (get link, apply code), and Utility (brand requests, admin approve/reject). Deal claiming uses Prisma's interactive transactions with serializable isolation to prevent race conditions.

PostgreSQL Database (Supabase)

The PostgreSQL database stores all persistent application state across four core tables: users, deals, claims, and brand_requests. The schema enforces referential integrity through foreign key constraints, prevents duplicate claims through a unique composite index on (user_id, deal_id), and uses ENUM types for constrained values (reward_type, claim status). Deals are seeded into the database via a Prisma seed script rather than created through a web portal, keeping the MVP scope manageable. Database access is exclusively mediated through the Prisma ORM.

Object Storage (AWS S3 + CloudFront)

The object storage layer handles proof-of-hangout photographs. The iOS app uploads directly to S3 using time-limited pre-signed URLs generated by the API, eliminating the API server as a bottleneck. CloudFront CDN caches and serves images with low latency. Image S3 URLs are stored as references in the claims table (proof_image_url column). Brand logos and deal images are also stored in S3 and referenced in the deals table.

6.2.2 Major Software Interactions

All communication between the iOS app and Backend API occurs over HTTPS using RESTful conventions. Requests carry JSON payloads and include a JWT Bearer token in the Authorization header for authenticated endpoints. The API responds with a consistent JSON envelope: { data: T, error: null } for success or { data: null, error: { code: string, message: string } } for errors, accompanied by standard HTTP status codes (200 success, 201 created, 400 validation error, 401 unauthorized, 404 not found, 409 conflict for duplicate claims, 500 server error).

The iOS app's APIClient singleton wraps Alamofire with a custom RequestInterceptor that injects the JWT token from KeychainManager into every request and handles 401 responses by attempting token refresh or triggering logout. All API methods are async/await and throw typed HangfulAPIError values for structured error handling in ViewModels.

For proof photo uploads, the iOS app follows a three-step process: (1) requests a pre-signed PUT URL from the API via GET /api/uploads/presign; (2) uploads the compressed JPEG directly to S3 via HTTP PUT to the pre-signed URL with a progress callback; (3) notifies the API of the completed upload via POST /api/claims/:id/proof with the S3 object URL. This keeps large binary data off the API server.

The Backend API communicates with PostgreSQL through Prisma's query engine with connection pooling via PgBouncer. The deal claiming operation is the most critical interaction: it uses Prisma's $transaction API with serializable isolation to atomically read the deal's claimed_quantity, validate availability (claimed_quantity < total_quantity), increment the count, and create the claim record. If any step fails or a concurrent claim races, the transaction rolls back and the client receives a 409 Conflict response.

The Backend API communicates with Twilio over HTTPS using the official twilio Node.js SDK. OTP calls are synchronous (blocking the auth response), while SMS reminders are fired asynchronously after claim creation using a simple setTimeout-based scheduler (sufficient for MVP scale; a Redis-backed job queue like Bull is planned for production scaling).

6.2.3 Architectural Design Diagrams

Use Case Diagram

The use case diagram shows the single primary actor (User on iOS) and their eight interactions with the Hangful system. Dashed arrows indicate dependency relationships: claiming a deal requires authentication, sharing a referral requires a completed proof, and viewing a redemption code requires admin approval.

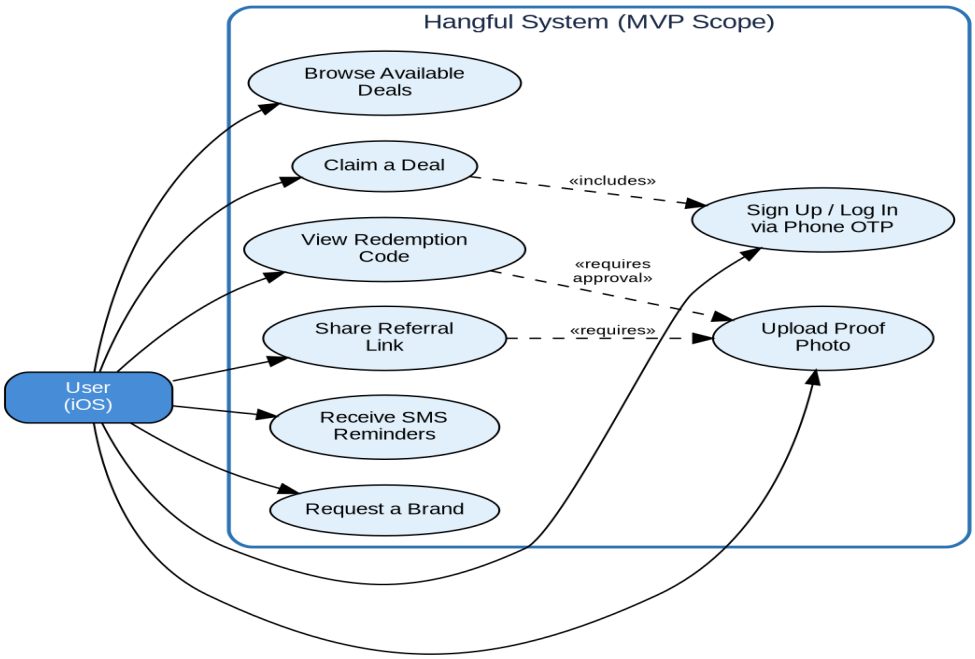

Figure 1: Hangful MVP Use Case Diagram

Component and Deployment Diagram

The component diagram shows the two-tier deployment architecture: the iOS client communicates with the Backend API over HTTPS/REST, and directly with S3 for image uploads via pre-signed URLs. The API layer contains four internal services (Auth Middleware, Upload Service, SMS Service) and connects to PostgreSQL via Prisma ORM and to Twilio for OTP and SMS.

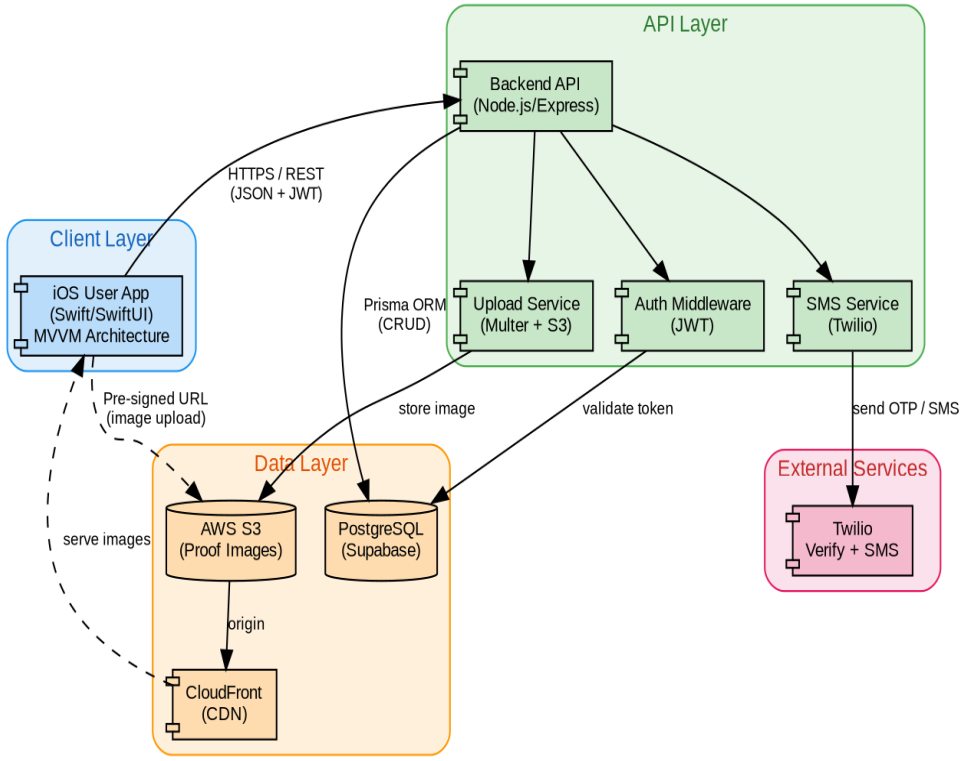

Figure 2: Component and Deployment Diagram

Core User Journey Sequence

The sequence diagram traces the eight-step user journey from authentication through redemption, identifying the specific API endpoint at each step. Note that step 7 (admin approval) occurs outside the iOS app via a database script or admin API call, reflecting the MVP's scope decision to defer the Brand Portal.

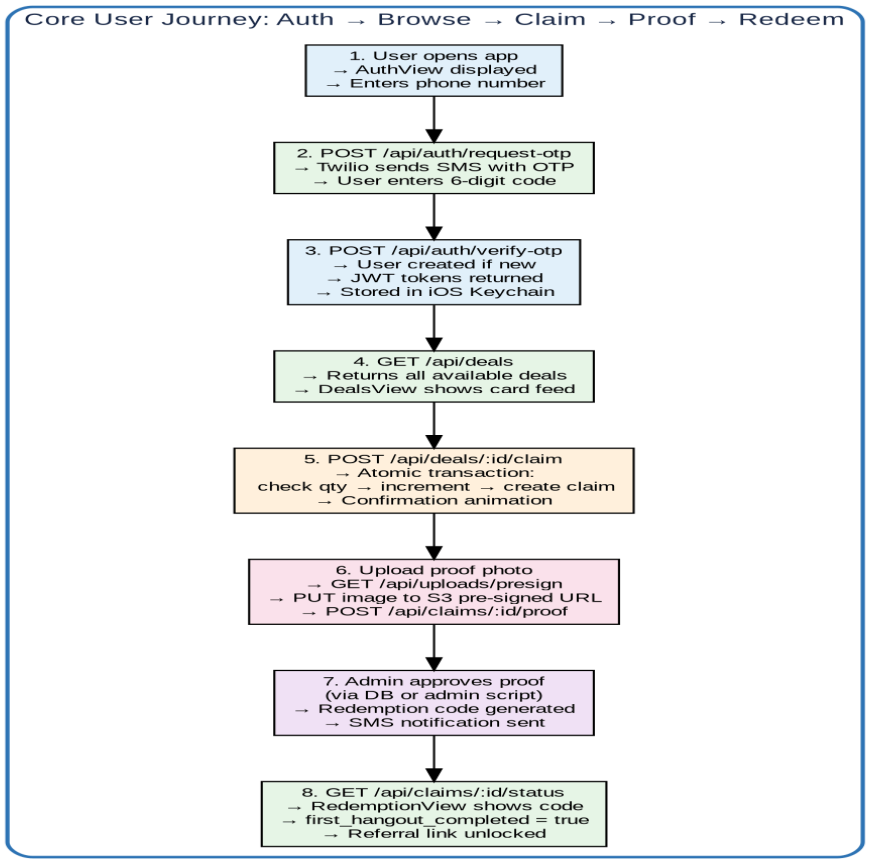

Figure 3: Core User Journey Sequence Diagram

Activity / Swimlane Diagram

The swimlane diagram partitions the proof-to-redemption workflow across four responsibility zones: User (iOS App), Backend API + Storage, Admin Approval (database script or future portal), and Redemption and Referral. This illustrates how the rejection-resubmission loop works and how the referral unlock is gated behind a completed hangout.

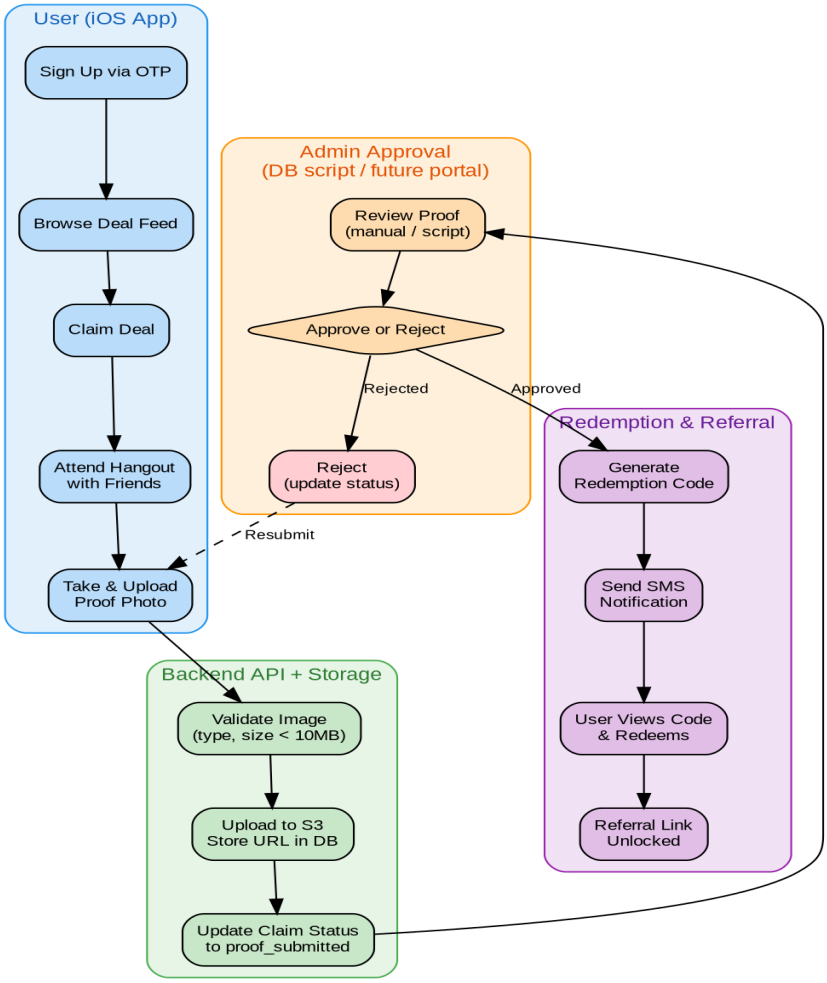

Figure 4: Proof-to-Redemption Swimlane Diagram

6.3 CSC and CSU Descriptions

The Hangful iOS App (CSCI) is decomposed into five Computer Software Components (CSCs): Models CSC, Views CSC, ViewModels CSC, Services CSC, and Utilities CSC. Each CSC contains multiple Computer Software Units (CSUs) corresponding to individual Swift files. The Backend API follows an analogous structure (routes, controllers, services, middleware, models) but its detailed design is described through its API contract and database interactions rather than class-level decomposition, since it is a solo project and the iOS app is the primary deliverable for the course. The following sections describe each CSU in detail, ordered from smallest to largest class within each CSC.

6.3.1 Class Descriptions

The following sections provide the details of all classes used in the Hangful application.

6.3.1.1 Models CSC: BrandRequest

The BrandRequest model captures user-submitted suggestions for brands they want to see on Hangful. It is the simplest model in the system, serving a data-collection purpose with no complex business logic. It is used by RequestBrandView and sent to the API via APIClient.requestBrand(name).

Field	Type	Description
id	UUID	Unique identifier; primary key
userId	UUID	Foreign key to the user who submitted the request
brandName	String	Name of the requested brand as entered by the user
createdAt	Date	Timestamp of submission

6.3.1.2 Models CSC: User

The User model represents an authenticated user of the Hangful iOS app. It is a Swift Codable struct that maps to the users table in PostgreSQL. The firstHangoutCompleted flag is the key gating mechanism for the referral feature—it is set to true server-side when a user's first claim transitions to the approved state.

Field	Type	Description
id	UUID	Unique identifier generated by the database; primary key
phoneNumber	String	User's phone in E.164 format; used for OTP auth; unique and indexed for fast lookup
displayName	String?	Optional display name shown in referral contexts; defaults to nil until set
referralCode	String	Auto-generated unique 8-character alphanumeric code for referral link construction (hangful.app/r/{code})
referredBy	UUID?	FK to the referring user's id; nil if organic signup; set during onboarding via applyReferral()

firstHangoutCompleted	Bool	Gate flag for referral feature; set true server-side on first proof approval
createdAt	Date	Account creation timestamp
updatedAt	Date	Most recent profile update timestamp

6.3.1.3 Models CSC: Deal

The Deal model represents a promotional offer from a brand. In the MVP, deals are always visible (no drop-based scheduling) and are seeded into the database rather than created through a web portal. The brand_name and brand_logo_url fields are stored directly on the deal rather than via a foreign key to a separate brands table, simplifying the schema for the MVP. Remaining quantity is computed client-side as totalQuantity minus claimedQuantity.

Field	Type	Description
id	UUID	Unique identifier; primary key
title	String	Deal headline displayed on card (e.g., “BOGO Boba Tea”)
description	String	Full deal terms and conditions shown on DealDetailView
brandName	String	Business name displayed on card; stored directly rather than via FK for MVP simplicity
brandLogoUrl	String?	S3/CDN URL of brand logo; displayed on deal cards alongside title
rewardType	RewardType	Enum: .discount, .freebie, or .credit; determines redemption UX
rewardValue	String	Human-readable reward (e.g., “50% off”, “Free appetizer”)
totalQuantity	Int	Maximum claims allowed; enforced atomically in database transaction
claimedQuantity	Int	Current claim count; incremented in transaction; default 0
expiresAt	Date	Timestamp after which deal cannot be claimed or redeemed
createdAt	Date	Timestamp of deal creation (seed time)

Key methods: isAvailable() returns true if claimedQuantity < totalQuantity and current time is before expiresAt. getRemainingQty() returns totalQuantity - claimedQuantity for display on the deal card.

6.3.1.4 Models CSC: Claim

The Claim model is the most complex entity in the system, representing the full lifecycle of a user’s interaction with a deal. The status ENUM field drives a state machine governing what actions are available at each stage. This model is used extensively by ClaimsViewModel and is the core data structure for MyClaimsView, ProofUploadView, and RedemptionView.

Field	Type	Description
id	UUID	Unique identifier; primary key
userId	UUID	FK to user who made this claim
dealId	UUID	FK to claimed deal; unique with userId to prevent double claims
status	ClaimStatus	Enum: .claimed, .proofSubmitted, .approved, .rejected, .redeemed
proofImageUrl	String?	S3/CDN URL of uploaded proof photo; nil until proof submitted
redemptionCode	String?	8-char crypto-random alphanumeric code; nil until admin approves
claimedAt	Date	Timestamp of initial claim action
submittedAt	Date?	Timestamp of proof upload; nil until submitted
approvedAt	Date?	Timestamp of admin approval; nil until approved
redeemedAt	Date?	Timestamp of code use at business; nil until redeemed

Key methods: submitProof(imageUrl) transitions status from .claimed to .proofSubmitted and stores the S3 URL. approve() generates a redemption code via crypto.randomBytes, transitions to .approved, and triggers an SMS notification. reject(reason) transitions to .rejected. generateCode() produces a unique 8-character alphanumeric string verified against existing codes.

Claim Lifecycle State Diagram

The state diagram below illustrates all valid transitions for a Claim, driven by the status ENUM field. The rejection-resubmission loop allows users to re-upload proof, and the expired terminal state handles deals that time out before proof submission.

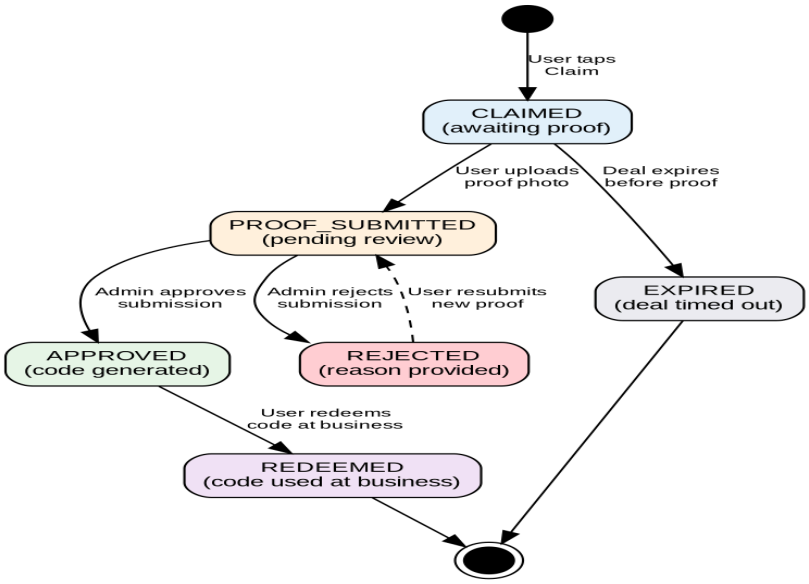

Figure 5: Claim Lifecycle State Diagram

6.3.1.5 ViewModels CSC: AuthViewModel

AuthViewModel is an ObservableObject class managing the authentication state machine. It exposes @Published properties: authState (enum: .unauthenticated, .otpSent, .authenticated), phoneNumber (String, user input), otpCode (String, 6-digit input), isLoading (Bool), and errorMessage (String?). It provides requestOTP() which calls AuthService.requestOTP and transitions to .otpSent, and verifyOTP() which calls AuthService.verifyOTP, stores the returned User, and transitions to .authenticated. On app launch, it checks KeychainManager for a valid stored token and auto-authenticates if found.

6.3.1.6 ViewModels CSC: DealsViewModel

DealsViewModel is an ObservableObject responsible for the deal feed. It exposes @Published properties: deals ([Deal], the full list of available deals), isLoading (Bool), errorMessage (String?), and isRefreshing (Bool, for pull-to-refresh). It calls APIClient.getDeals() on initialization and on refresh, parsing the JSON response into Deal model arrays. Since deals are always available in the MVP (no drops), there is no countdown timer logic. Deals with claimedQuantity >= totalQuantity are shown with a “Sold Out” badge rather than hidden.

6.3.1.7 ViewModels CSC: ClaimsViewModel

ClaimsViewModel manages the user’s claims list and proof submission workflow. It exposes @Published properties: claims ([Claim], sorted by status then date), selectedClaim (Claim?, for proof upload context), uploadProgress (Double, 0.0–1.0), and proofImage (UIImage?, captured photo). Methods: claimDeal(dealId) calls the claim API and inserts the new claim into the local array; submitProof(claimId, imageData) compresses the image, calls ImageUploader.upload, and updates the claim status locally; refreshClaims() polls GET /api/claims for updated statuses. The ViewModel also exposes a computed property pendingProofCount for badge display.

6.3.1.8 ViewModels CSC: ReferralViewModel

ReferralViewModel manages the referral feature state. It exposes @Published properties: referralLink (String?, nil if ineligible), isEligible (Bool, bound to User.firstHangoutCompleted), and referralCount (Int, number of users referred). On initialization, it calls APIClient.getReferralLink(). The shareReferral() method constructs a share message and presents UIActivityViewController for sharing via iMessage, WhatsApp, or other apps. When isEligible is false, ReferralView displays a locked state explaining the user must complete their first hangout.

6.3.1.9 Services CSC: KeychainManager

KeychainManager is the smallest service class, providing a typed wrapper around KeychainSwift for secure credential storage. Methods: saveAccessToken(token: String), getAccessToken() -> String?, saveRefreshToken(token: String), getRefreshToken() -> String?, and clearAll(). All values use kSecAttrAccessibleWhenUnlockedThisDeviceOnly to prevent extraction from backups or other devices. This class is used by AuthService for token persistence and by APIClient's request interceptor for token retrieval.

6.3.1.10 Services CSC: AuthService

AuthService encapsulates authentication API calls and token lifecycle management. Methods: requestOTP(phoneNumber: String) async throws calls POST /api/auth/request-otp; verifyOTP(phoneNumber: String, code: String) async throws -> User calls POST /api/auth/verify-otp, parses the response, stores both tokens via KeychainManager, and returns the authenticated User; refreshToken() async throws -> Bool exchanges the refresh token for a new access token; logout() clears all tokens via KeychainManager.clearAll() and resets the app to AuthView.

6.3.1.11 Services CSC: ImageUploader

ImageUploader handles the three-step proof image upload process. Its primary method uploadProofImage(imageData: Data, claimId: UUID) async throws -> String performs: (1) compress UIImage to JPEG at 0.7 quality, validate size under 10MB; (2) call APIClient to get a pre-signed S3 PUT URL (GET /api/uploads/presign?contentType=image/jpeg); (3) perform HTTP PUT to the pre-signed URL with the image data, publishing upload progress via a Combine PassthroughSubject<Double, Never>; (4) return the permanent S3 object URL on success. Failed uploads are retried up to 3 times with exponential backoff (1s, 3s, 9s).

6.3.1.12 Services CSC: APIClient

APIClient is the largest service class, serving as the centralized HTTP client for all backend communication. It is a singleton (APIClient.shared) wrapping an Alamofire Session with a custom RequestInterceptor that injects the JWT Bearer token from KeychainManager and handles 401 responses by calling AuthService.refreshToken() before retrying. It exposes typed async methods: getDeals() -> [Deal]; claimDeal(id: UUID) -> Claim; getClaims() -> [Claim]; getClaimStatus(id: UUID) -> Claim; getPresignedURL(contentType: String) -> PresignedURLResponse; submitProof(claimId: UUID, imageUrl: String) -> Claim; getReferralLink() -> ReferralResponse; applyReferral(code: String) -> Bool; requestBrand(name: String) -> BrandRequest. All methods throw HangfulAPIError with cases: .unauthorized, .notFound, .conflict(String), .validationError(String), .serverError, .networkError.

6.3.2 Detailed Interface Descriptions

iOS App to Backend API Interface

All iOS-to-API communication uses HTTPS REST with JSON request/response bodies. The base URL is configured per Xcode scheme: debug uses http://localhost:3000/api, staging uses https://api-staging.hangful.app/api, and production uses https://api.hangful.app/api. The APIClient's Alamofire RequestInterceptor automatically attaches Authorization: Bearer <token> to every request. On 401 responses, the interceptor attempts one token refresh; if that fails, it calls AuthService.logout() and redirects to AuthView. All responses conform to the envelope { data: T | null, error: { code: string, message: string } | null }.

iOS App to S3 Interface

Proof uploads bypass the API server entirely. The iOS app calls GET /api/uploads/presign?contentType=image/jpeg to obtain a pre-signed PUT URL valid for 15 minutes. It then performs a raw HTTP PUT (via Alamofire's upload method) to the S3 URL with the JPEG data and Content-Type header. On success (HTTP 200), the permanent object URL (https://cdn.hangful.app/proofs/{uuid}.jpg) is sent to POST /api/claims/:id/proof.

Backend API to PostgreSQL Interface

All database communication goes through Prisma ORM with parameterized queries (preventing SQL injection). Connection pooling is handled by PgBouncer with a max of 20 connections. The deal claim operation uses $transaction with serializable isolation for atomicity. The connection string is injected via the DATABASE_URL environment variable with sslmode=require.

Backend API to Twilio Interface

Twilio communication uses the official twilio Node.js SDK (v4.x). OTP operations call client.verify.v2.services(SID).verifications.create (to send) and .verificationChecks.create (to verify). SMS reminders call client.messages.create with the destination phone, a templated message body, and the Twilio phone number as sender. Credentials are injected via TWILIO_ACCOUNT_SID and TWILIO_AUTH_TOKEN environment variables.

6.3.3 Detailed Data Structure Descriptions

API Response Envelope

All API responses use: { data: T, error: null } for success or { data: null, error: { code: string, message: string } } for errors. Error codes are machine-readable identifiers: INVALID_OTP, OTP_RATE_LIMITED, DEAL_SOLD_OUT, DEAL_EXPIRED, ALREADY_CLAIMED, CLAIM_NOT_FOUND, REFERRAL_NOT_ELIGIBLE, INVALID_REFERRAL_CODE, INVALID_IMAGE_TYPE, IMAGE_TOO_LARGE. HTTP status codes: 200/201 success, 400 validation, 401 auth, 404 not found, 409 conflict, 429 rate limited, 500 server error.

JWT Token Payload

Access tokens: { sub: UUID (user id), iat: number, exp: number (15 min from issue) }, signed with HS256 using the JWT_SECRET environment variable. Refresh tokens: { sub: UUID, type: "refresh", iat: number, exp: number (7 days) }. The sub field is used by auth middleware to load the user from the database on each request.

Pre-signed URL Response

GET /api/uploads/presign returns: { uploadUrl: string (time-limited S3 PUT URL), objectUrl: string (permanent CDN URL), expiresIn: 900 (seconds) }. The uploadUrl contains the S3 bucket,

key (proofs/{uuid}.jpg), and pre-signed query parameters. The objectUrl maps to the CloudFront distribution and is the value stored in claims.proof_image_url.

HangfulAPIError Enum

The iOS app's typed error enum: case unauthorized (token invalid/expired, triggers logout), case notFound (resource missing), case conflict(String) (duplicate claim, deal sold out), case validationError(String) (bad input), case serverError (500), case networkError (no connectivity). ViewModels map these to user-facing error messages displayed via alert dialogs.

6.3.4 Detailed Design Diagrams

Detailed Class Diagram

The class diagram shows all four model classes with complete field definitions, data types, method signatures, and relationship multiplicities. The self-referential User relationship tracks the referral chain.

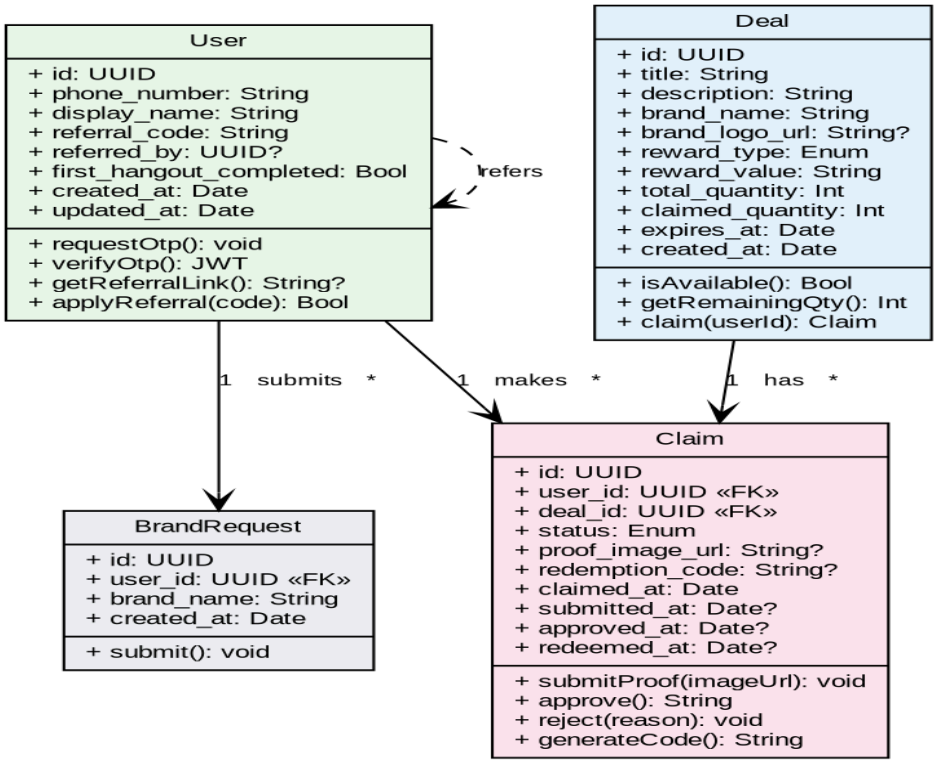

Figure 6: Detailed Class Diagram with Fields and Methods

Package Diagram (CSCI Decomposition)

The package diagram shows the iOS app's decomposition into five CSCs (Models, Views, ViewModels, Services, Utilities) with dependency arrows between individual CSUs. The unidirectional flow (Views → ViewModels → Services → Models) confirms clean MVVM separation with no circular dependencies.

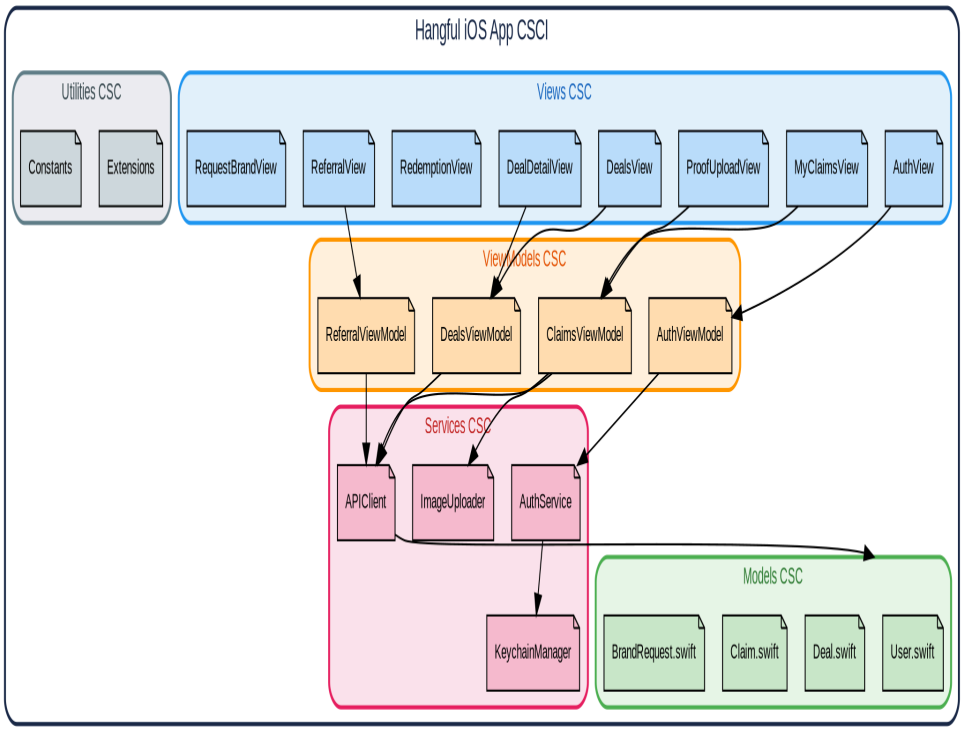

Figure 7: iOS App Package Diagram (CSCI → CSC → CSU)

6.4 Database Design and Description

The Hangful MVP uses PostgreSQL 16 hosted on Supabase as its relational database. The schema consists of four tables storing user accounts, deals, claims, and brand requests. Deals are populated via a Prisma seed script (npx prisma db seed) rather than a web portal, keeping administration simple for the MVP.

6.4.1 Database Design ER Diagram

The entity-relationship diagram shows all four tables with their columns, data types, primary keys (PK), unique keys (UK), foreign keys (FK), and cardinalities in crow's foot notation. The self-referential relationship on USERS (referred_by → id) tracks referral chains. The UNIQUE constraint on CLAIMS(user_id, deal_id) prevents a user from claiming the same deal twice.

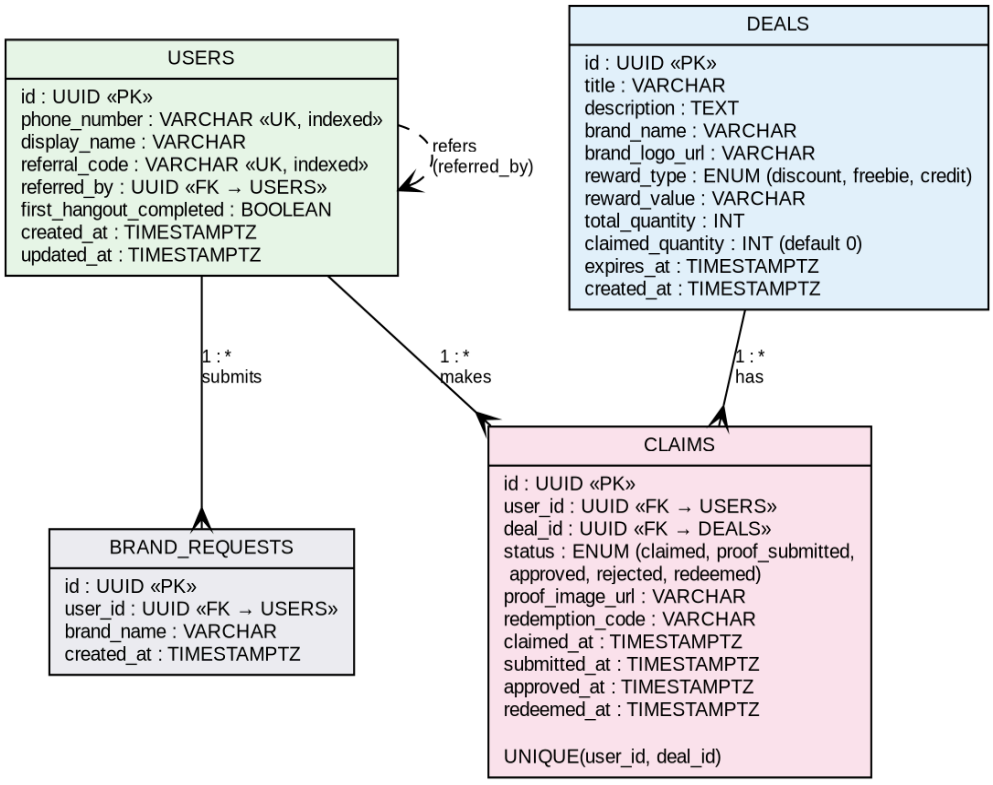

Figure 8: Database Entity-Relationship Diagram (4 Tables)

Key design decisions: UUID primary keys (via gen_random_uuid()) provide collision-free, non-guessable identifiers. Brand information is denormalized onto the deals table (brand_name, brand_logo_url) rather than using a separate brands table with a foreign key, eliminating a join on every deal feed query and simplifying the schema for a solo MVP. ENUM types enforce constrained values at the database level for reward_type (discount, freebie, credit) and claim status (claimed, proof_submitted, approved, rejected, redeemed). All timestamps use TIMESTAMPTZ for timezone-safe storage.

Indexes

Index	Column(s)	Purpose
idx_users_phone	users.phone_number (UNIQUE)	Fast OTP auth lookup; prevents duplicate accounts
idx_users_referral	users.referral_code (UNIQUE)	Fast referral code validation during onboarding
idx_claims_user_deal	claims(user_id, deal_id) UNIQUE	Prevents double claims; fast lookup for claim status checks
idx_claims_deal_status	claims(deal_id, status)	Efficient filtering for admin proof review by deal

6.4.2 Database Access

All database access is mediated through the Prisma ORM, which generates a type-safe client from the schema.prisma file. No raw SQL queries are used; all operations go through Prisma's query builder with automatically parameterized inputs, preventing SQL injection.

Read operations use Prisma's findMany (deal feed: SELECT * FROM deals WHERE expires_at > NOW() ORDER BY created_at DESC), findUnique (claim status lookup by ID), and findFirst (user lookup by phone_number during auth). The deal feed query is the most frequent read path and benefits from the timestamp index on expires_at.

The deal claiming write path is the most performance-critical operation and uses Prisma's interactive transaction ($transaction) with serializable isolation. Within the transaction: (1) SELECT the deal's claimed_quantity with a row-level lock (FOR UPDATE); (2) validate claimed_quantity < total_quantity; (3) UPDATE deals SET claimed_quantity = claimed_quantity + 1; (4) INSERT into claims with status 'claimed'. If step 2 fails, the transaction rolls back and the API returns 409 Conflict with error code DEAL_SOLD_OUT.

Deal seeding is performed via a Prisma seed script (seed.ts) invoked by npx prisma db seed. The seed file contains an array of deal objects with all required fields, allowing rapid iteration on test data during development and populating the production database with launch partner deals.

6.4.3 Database Security

Network isolation: The Supabase PostgreSQL instance is configured with network-level access controls. Only the Railway-hosted API server's IP addresses are whitelisted. No direct client-to-database connections are permitted; all access routes through the API which enforces JWT authentication on every request.

Encrypted transport: All API-to-database connections use SSL/TLS (sslmode=require in the connection string), preventing data interception during transmission.

Credential management: The DATABASE_URL connection string (containing credentials, host, and database name) is stored exclusively in Railway environment variables and is never committed to source control. The .env file is listed in .gitignore.

Input sanitization: Prisma's query builder automatically parameterizes all user-supplied values, eliminating SQL injection as an attack vector. No raw SQL is used anywhere in the codebase.

Row-level data isolation: The JWT auth middleware extracts the user's UUID from the token, and all queries are scoped to the authenticated user. For example, GET /api/claims returns only rows WHERE user_id = authenticatedUser.id. A user cannot access another user's claims, referral data, or brand requests.

Backup and recovery: Supabase provides automated daily database backups with point-in-time recovery (PITR). Backups are encrypted at rest using AES-256 with a 7-day retention period on the free tier.